

Sistemas Operativos de Tiempo Real (3)

Técnicas Digitales III

Tareas

- Las tareas se implementan como funciones de lenguaje C.
- Deben retornar un tipo void y deben tomar un puntero void como parámetro.

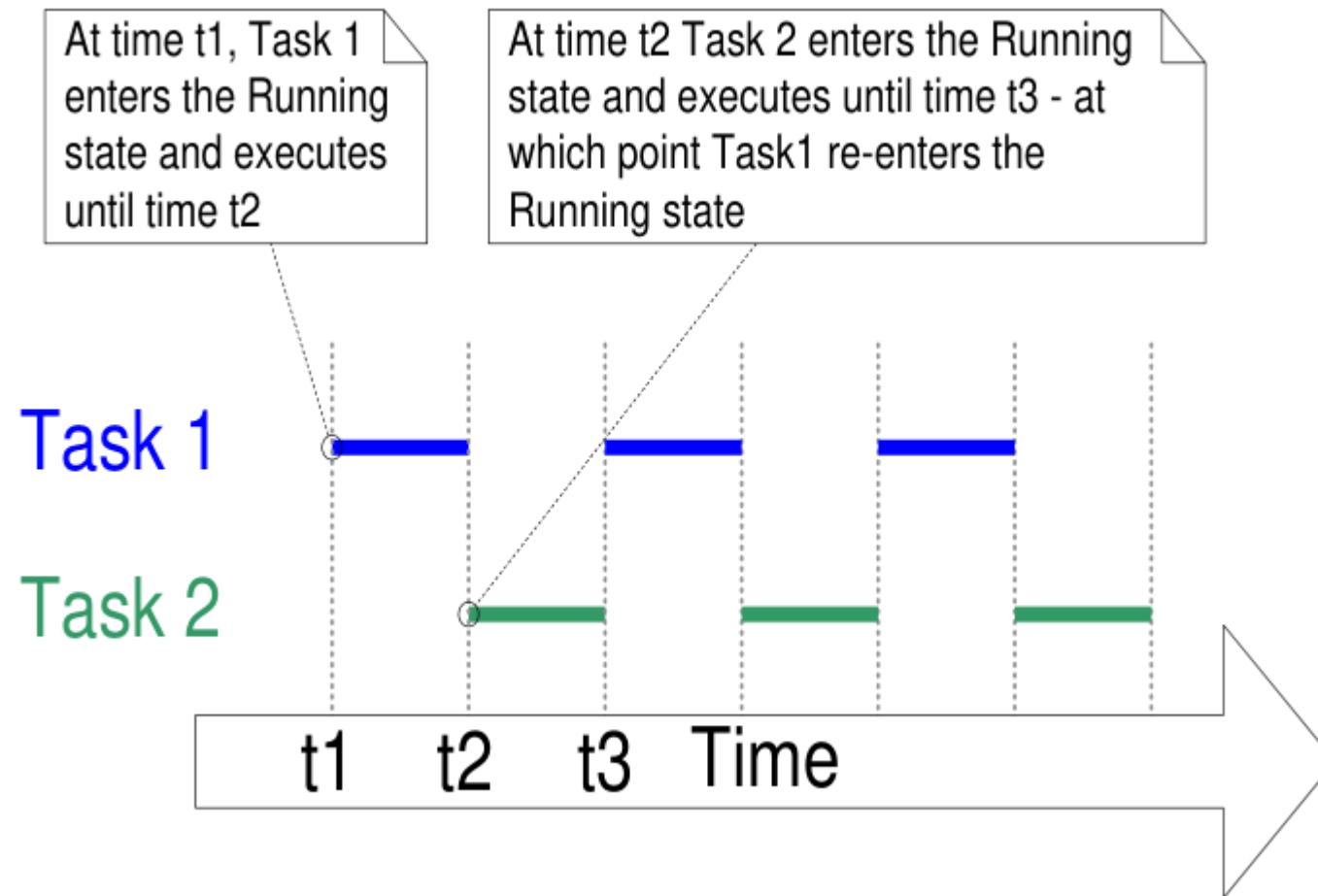
```
void ATaskFunction( void *pvParameters );
```

Tareas

- Las tareas tienen un punto de entrada y se ejecutan por siempre en un bucle infinito.

```
void ATaskFunction( void *pvParameters )
{
    int32_t lVariableExample = 0;
    /* A task will normally be implemented as an infinite loop. */
    for( ;; )
    {
        /* The code to implement the task functionality will go here. */
    }
    vTaskDelete( NULL );
}
```

Tareas



Tareas

- Es posible crear una tarea desde otra tarea:

```
void vTask1( void *pvParameters )
{
    const char *pcTaskName = "Task 1 is running\r\n";
    volatile uint32_t ul;
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );
    for( ;; )
    {
        vPrintString( pcTaskName );
        for( ul = 0; ul < mainDELAY_LOOP_COUNT; ul++ )
        {
        }
    }
}
```

Tareas

- Es posible crear una tarea desde otra tarea:

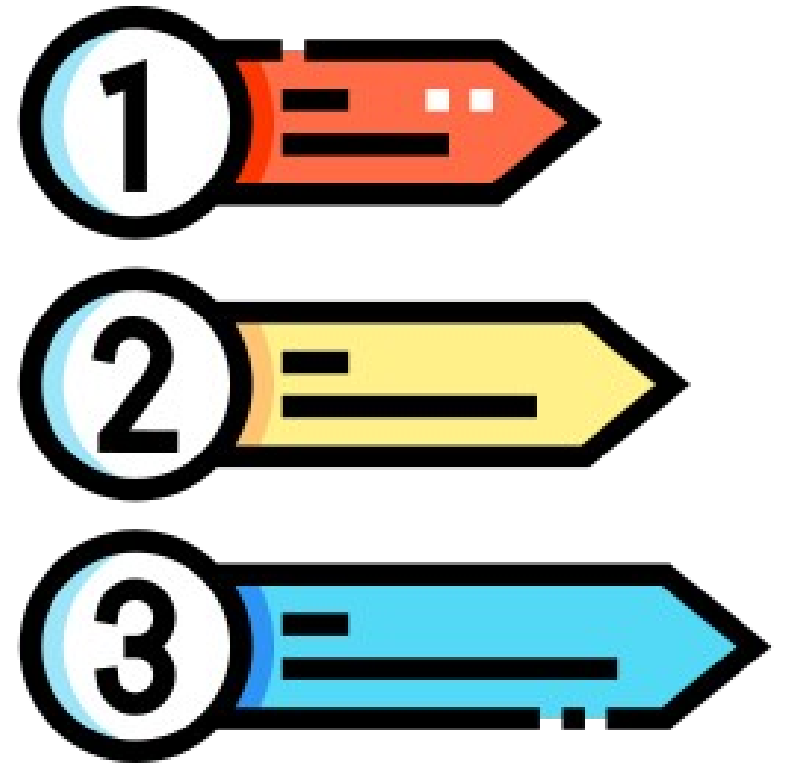
```
static const char *pcTextForTask1 = "Task 1 is running\r\n";
static const char *pcTextForTask2 = "Task 2 is running\r\n";
int main( void )
{
    xTaskCreate( vTaskFunction, "Task 1", 1000, (void*)pcTextForTask1, 1, NULL );
    xTaskCreate( vTaskFunction, "Task 2", 1000, (void*)pcTextForTask2, 1, NULL );

    vTaskStartScheduler();

    For( ;; );
}
```

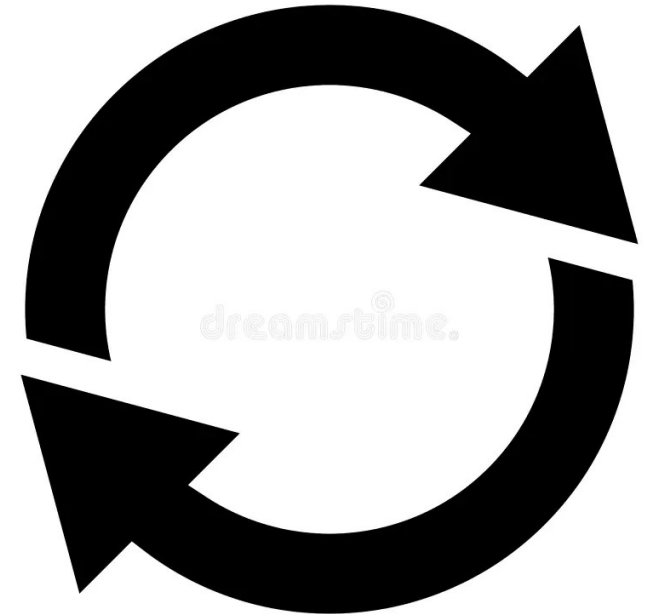
Tareas

- Las prioridades se definen en `configMAX_PRIORITIES` en tiempo de compilación en el archivo `FreeRTOSConfig.h`
- A mayor valor numérico de la prioridad, mayor prioridad.
- Las prioridades disponibles van de 0 a `configMAX_PRIORITIES-1`



Tareas

- En planificador puede utilizar dos métodos para determinar qué tarea coloca en ejecución:
 - Genérico (sin optimización).
 - Optimizado para la arquitectura (no siempre disponible).
- El planificador procurará que siempre se ejecute la tarea de mayor prioridad.



Timer Tick

- Si se utiliza “time slicing”, el planificador se ejecuta periódicamente.
- El período se ajusta mediante la constante `configTICK_RATE_HZ`.
- Por ejemplo, si se ajusta a 100Hz, el período será 10 ms

